

C++ 프로그래밍

(프렌드와 연산자 중복 실습)

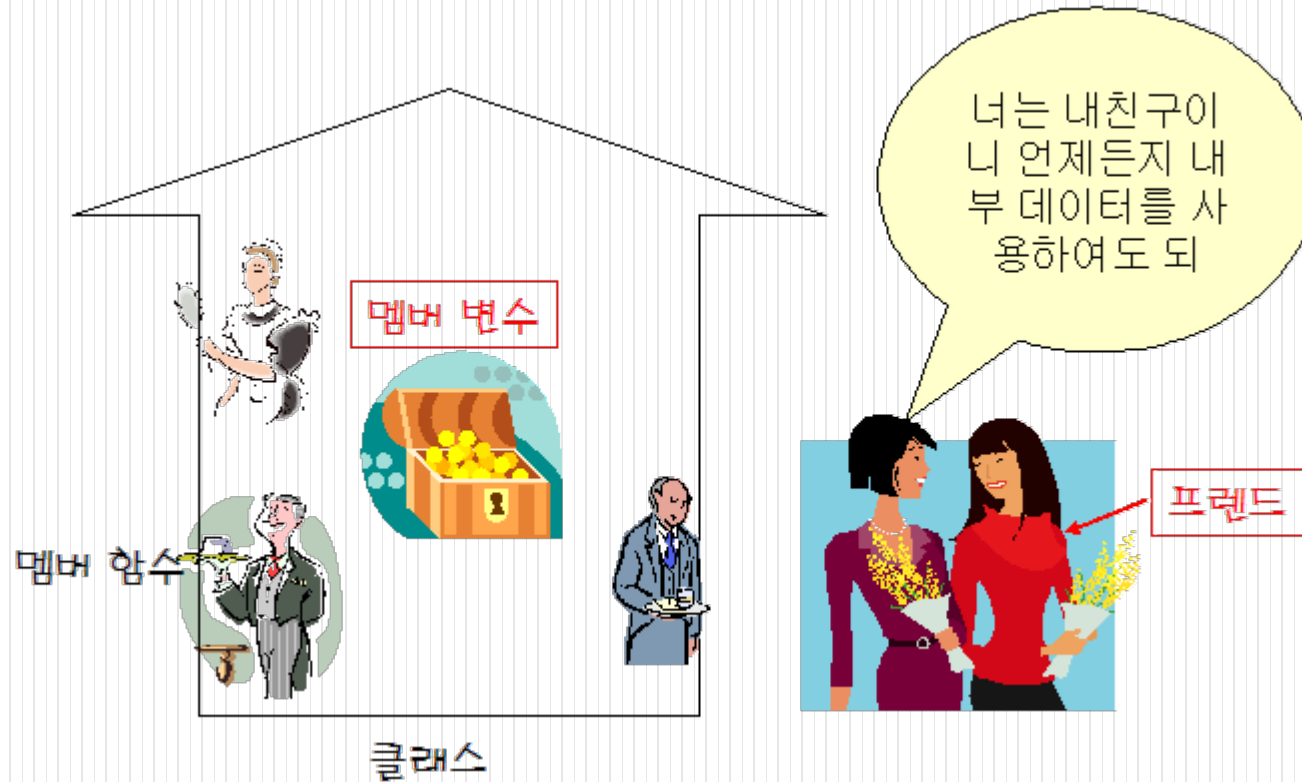
9주차

목차

- 프렌드 함수
- 연산자 중복
- 타입 변환
- 연산자 중복과 프렌드 함수 예제
 - MyString만들기
 - MyArray만들기

프렌드 함수

- 클래스의 내부 데이터에 접근할 수 있는 특수한 함수



프렌드 함수 (계속)

- 프렌드 함수 선언 방법
 - 프렌드 함수의 원형은 비록 클래스 안에 포함.
 - 하지만 멤버 함수는 아니다.
 - 프렌드 함수의 본체는 외부에서 따로 정의
 - 프렌드 함수는 클래스 내부의 모든 멤버 변수를 사용 가능

```
class MyClass
{
    friend void sub();
    ....
};
```

← 프렌드 함수

```
#include <iostream>
#include <string>
using namespace std;

class Company {
private:
    int sales, profit;

    // sub()는 Company의 전용부분에 접근할 수 있다.
    friend void sub(Company& c);

public:
    Company(): sales(0), profit(0)
    {
    }
};

void sub(Company& c)
{
    cout << c.profit << endl;
}
```

```
int main()
{
    Company c1;
    sub(c1);
    return 0;
}
```

0

프렌드 함수 (계속)

- 프렌드 클래스
 - 클래스도 프렌드로 선언할 수 있다.
 - Manager의 멤버들은 Employee의 전용 멤버를 직접 참조할 수 있다.

```
class Employee {  
    int salary;  
    // Manager는 Employee의 전용 부분에 접근할 수 있다.  
    friend class Manager;  
    // ...  
};
```

프렌드 클래스

프렌드 함수 (계속)

- 프렌드 함수의 용도
 - 두개의 객체를 비교할 때 많이 사용

① 일반 멤버 함수 사용

```
if( obj1.equals(obj2) )  
{  
    ...  
}
```

② 프렌드 함수 사용

```
if( equals(obj1, obj2) )  
{  
    ...  
}
```

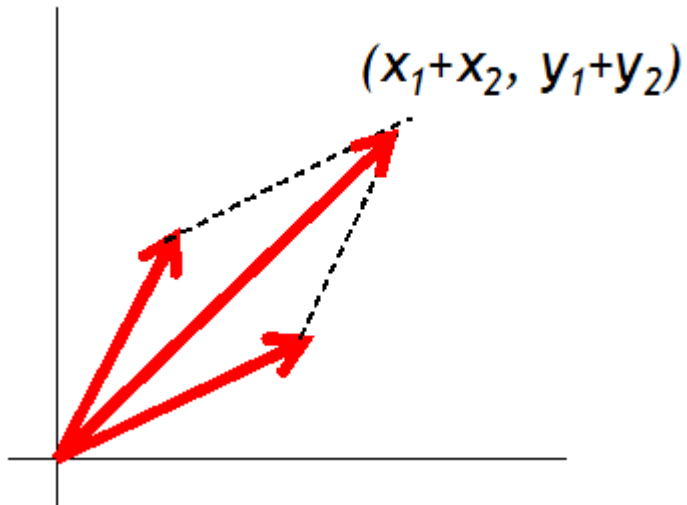
이해하기가 쉽다

연산자 중복

- 연산자 중복
 - 일반적으로는 연산자 기호를 사용하는 편이 함수를 사용하는 것보다 이해하기가 쉽다.
- 두 가지 문자 중에서 어떤 것이 더 하기 쉬운가?
 - `Sum = x+y_z;`
 - `Sum = add(x, add(y,z));`

연산자 중복 (계속)

- 벡터간의 연산을 연산자로 표기
 - vector v_1, v_2, v_3 ;
 - $v_3 = v_1 + v_2$;



연산자 중복 (계속)

- 연산자 중복(operator overloading)
 - 여러 가지 연산자들을 클래스 객체에 대해서도 적용하는 것
 - C++에서 연산자는 함수로 정의

```
반환형 operator연산자(매개 변수 목록)
{
    ....// 연산 수행
}
```

(예) Vector operator+(const Vector&, const Vector&);

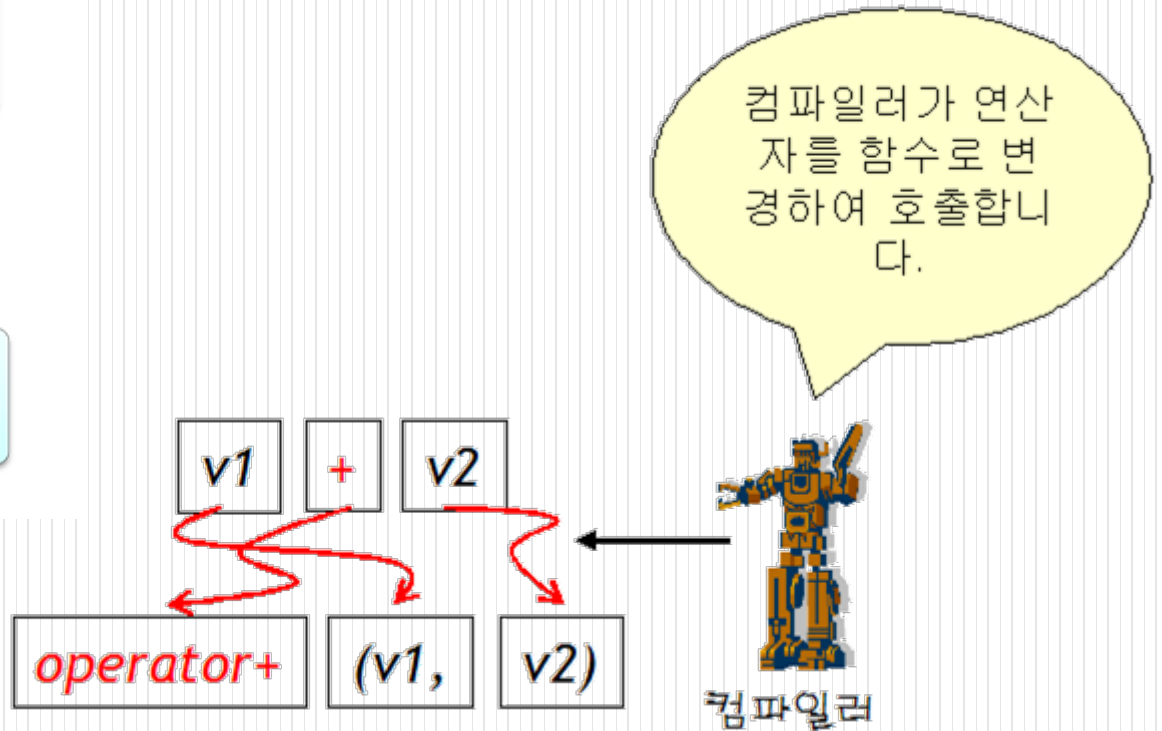
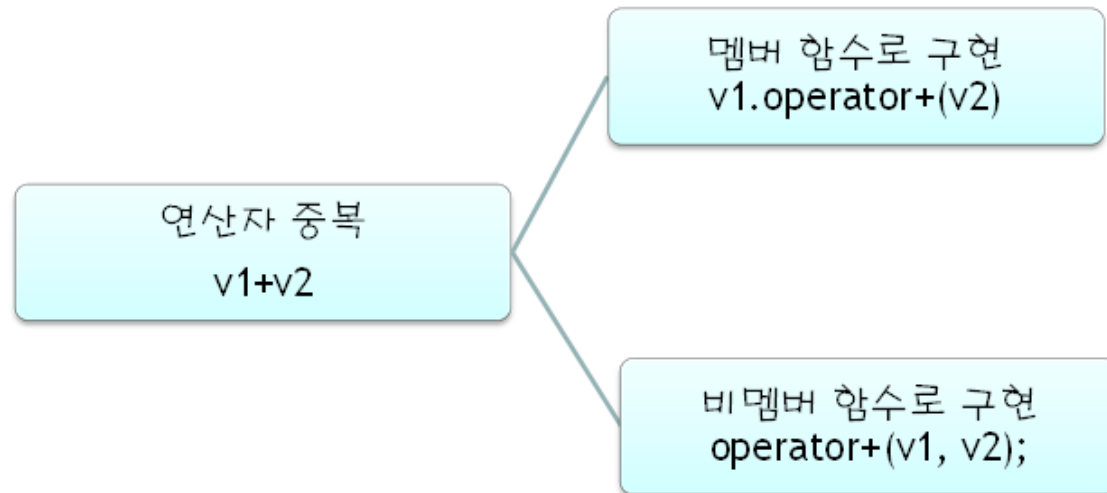
연산자 중복 (계속)

- 연산자 중복 함수 이름
 - 중복 함수 이름은 operator에 연산자를 붙이고 함수 기호

연산자	중복 함수 이름
+	operator+()
-	operator-()
*	operator*()
/	operator/()

연산자 중복 (계속)

- 연산자 중복 구현의 방법



연산자 중복 (계속)

- 멤버 함수로만 구현가능한 연산자
 - 연산자는 항상 멤버 함수 형태로만 중복 정의가 가능

연산자	설명
=	대입 연산자
()	함수 호출 연산자
[]	배열 원소 참조 연산자
->	멤버 참조 연산자

- 중복이 불가능한 연산자
 - 연산자는 중복 정의가 불가능

연산자	설명
::	범위 지정 연산자
.	멤버 선택 연산자
.*	멤버 포인터 연산자
?:	조건 연산자

연산자 중복 (계속)

- 곱셈 연산자 중복
 - 곱셈 연산자를 중복하여 정의
 - 교환 법칙이 성립하여야 함.

Vector operator*(Vector& v, double alpha); // $v * 2.0$ 형태 처리

Vector operator*(double alpha, Vector& v); // $2.0 * v$ 형태 처리

연산자 중복 (계속)

- "==" 연산자 중복
 - 두개의 객체가 동일한 데이터를 가지고 있는지를 체크하는데 사용

연산자	중복 함수 이름
==	operator==()
!=	operator!=()

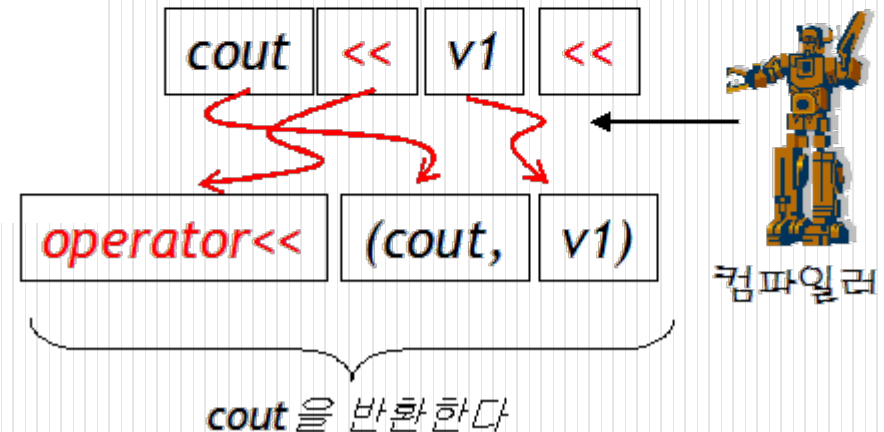
연산자 중복 (계속)

- "<<" 연산자의 중복
 - 연산을 수행한 후에 다시 스트림 객체를 반환하여야 함.

```
Vector v(2, 3);  
cout << v;           //화면에 (2, 3)이 출력된다.
```



어떤 객체든지
`cout << obj;`
하여 출력할수 있으면
편리하겠군



연산자 중복 (계속)

- ">>" 연산자의 중복
 - 입력 연산자 >>의 중복
 - 오류 처리를 하는 것이 좋음.
- "=" 연산자의 중복

```
istream& operator>>(istream& in, Vector& v)
{
    in >> v.x >> v.y;
    if(!in)
        v = Vector(0, 0);
    return in;
}
```

입력 오류 처리

```
class Vector
{
    ...
    Vector& operator=(const Vector& v2)
    {
        this->x = v2.x;
        this->y = v2.y;
        return *this;
    }
    ...
};
```

주의: 반드시 현재 객체의 레퍼런스를 반환

```
Vector v1(2.0, 3.0);
v3 = v2 = v1; // 가능!
```


연산자 중복 (계속)

- 증가/감소 연산자의 중복
 - ++와 --연산자의 중복

연산자	중복 함수 이름
++V	<u>v.operator++()</u>
--V	<u>v.operator--()</u>

- 전위/후의 문제
 - 전위와 후위 연산자를 구별하기 위하여 ++가 피연산자 뒤에 오는 경우에는 int형 매개 변수를 추가 한다.

연산자	중복 함수 이름
++V	<u>v.operator++()</u>
V++	<u>v.operator++(int)</u>

```
Vector& operator++()  
{  
    x++;  
    y++;  
    return *this;  
}
```

++v 형태의 증가 연산자
함수 정의

```
const Vector operator++(int)  
{  
    Vector saveObj = *this;  
    x++;  
    y++;  
    return saveObj;  
}  
};
```

v++ 형태의 증가 연산자
함수 정의

연산자 중복 (계속)

- [] 연산자 중복
 - 인덱스 연산자의 중복

연산자	중복 함수 이름
v[]	<u>v.operator[]()</u>

```
MyArray A;
```

```
A[3] = 10;
```

```
A.operator[](3) = 10;
```

- 포인터 연산자의 중복
 - 간접 참조 연산자 "*"와 멤버 연산자 "->"의 중복 정의

연산자	중복 함수 이름
*	operator*()
->	operator->()

연산자 중복 (계속)

- 스마트 포인터
 - 포인터 연산 정의를 이용하여서 만들어진 향상된 포인터를 스마트 폰터(smart pointer)라고 한다.
 - 주로 동적 할당된 공간을 반납할 때 사용된다.

```
class Pointer {  
    Car *pc;  
public:  
    Pointer(Car *p): pc(p){ }  
    ~Pointer(){ delete pc; }  
    Car* operator->() const { return pc; }  
    Car& operator*() const { return *pc; }  
};
```

스마트 포인터

```
int main()  
{  
    Pointer p(new Car(0,1,"red"));  
    p->speed = 100;  
    cout << *p;  
    (*p).speed = 200;  
    cout << *p;  
    p->setSpeed(300);  
    cout << *p;  
    return 0;  
}
```

연산자 중복

- 함수 호출 연산자()의 중복
 - 함수 호출때 사용하는 ()도 중복이 가능하다.

연산자	중복 함수 이름
<u>f(...)</u>	<u>f.operator()(...)</u>

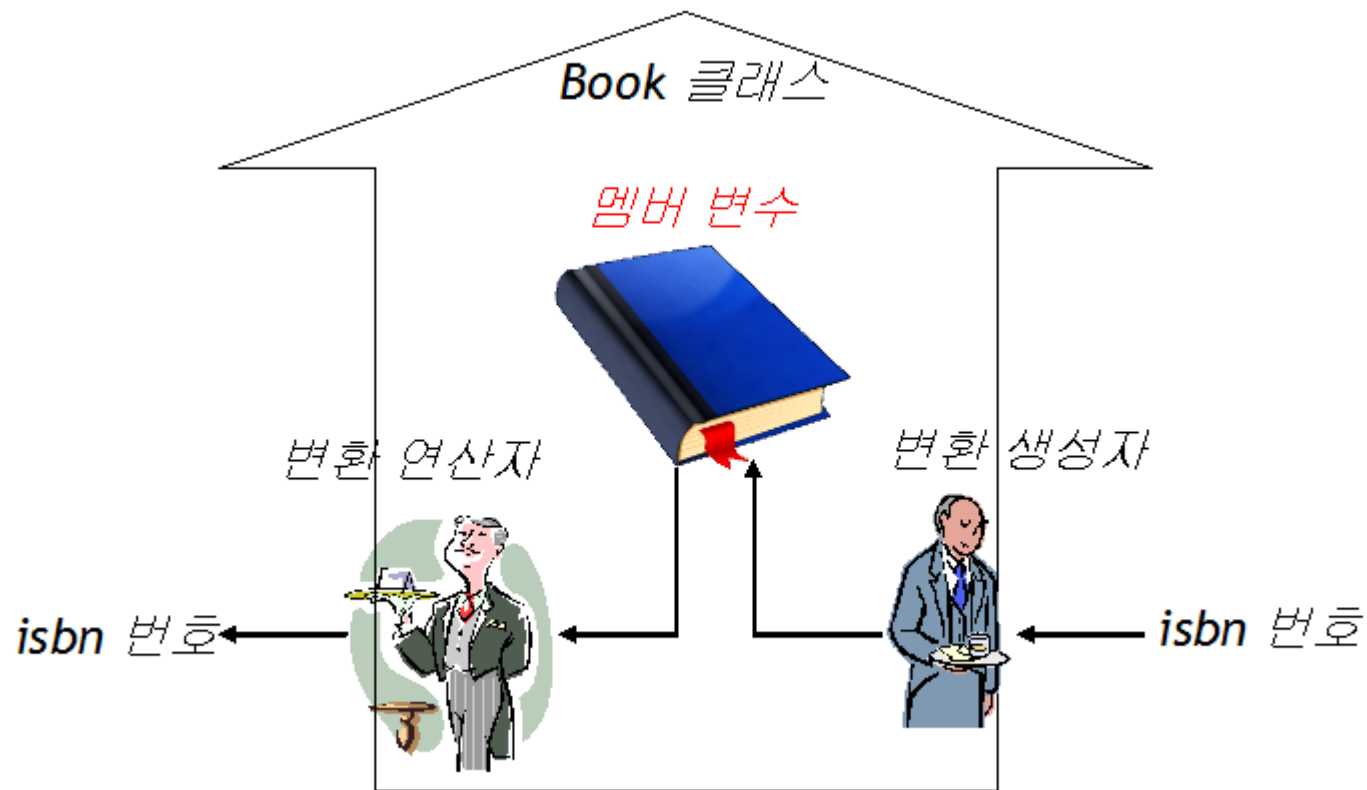
타입 변환

- 타입 변환
 - 클래스의 객체들도 하나의 타입에서 다른 타입으로 자동적인 변환이 가능하다.
 - 이것은 변환 생성자(conversion constructor)와 변환 연산자(conversion operator)에 의하여 가능하다.



타입 변환 (계속)

- 변환 생성자와 변환 연산자



타입 변환 (계속)

- 연산자 중복시 주의할 점
 - 새로운 연산자를 만드는 것은 허용되지 않는다.
 - "::" 연산자, "*" 연산자, "." 연산자, "?:" 연산자는 중복이 불가능하다.
 - 내장된 int형이나 double형에 대한 연산자의 의미를 변경할 수는 없다.
 - 연산자들의 우선 순위나 결합 법칙은 변경되지 않는다.
 - 만약 "+" 연산자를 오버로딩하였다면 일관성을 위하여 "+=", "-=" 연산자도 오버로딩하는 것이 좋다.
 - 일반적으로 산술 연산자와 관계 연산자는 비멤버 함수로 정의한다. 반면 할당 연산자는 멤버 함수로 정의한다.

연산자 중복과 프렌드 함수 예제

- 연산자 중복과 프렌드 함수를 이용한 MyString 만들기

```
int main() {  
  
    MyString s1("Hello ");  
    MyString s2("World!");  
  
    s1.print();  
    s2.print();  
    cout << "s1+s2=>" << (s1+s2) << endl;  
  
    return 0;  
}
```

```
Hello  
World!  
Hello World!
```


연산자 중복 예제 (계속)

```
#include <iostream>
#include <cstring>
using namespace std;

class MyString
{
    friend ostream& operator<<(ostream &, const MyString &);
private:
    char *pBuf;           //동적으로할당된메모리의주소값저장

public:
    MyString(const char *s=NULL);
    MyString(MyString& s);
    ~MyString();

    void print();         // 문자열을화면에출력
    int getSize();        // 문자열의길이반환
    MyString operator+(MyString& s);    // + 연산자중복정의
};
```

```
// 생성자
MyString::MyString(const char *s)
{
    if( s == NULL )
    {
        pBuf = new char[1];
        pBuf[0] = NULL;
    }
    else
    {
        pBuf = new char[sizeof(s)+1];
        memset(pBuf, 0x00, sizeof(s)+1);
        memcpy(pBuf, s, sizeof(s));
    }
}

// 복사생성자
MyString::MyString(MyString &s)
{
    pBuf = new char[s.getSize()+1];
    memset(pBuf, 0x00, s.getSize()+1);
    memcpy(pBuf, s.pBuf, s.getSize());
}
```

```
MyString::~MyString()
{
    if ( pBuf )
        delete [] pBuf;
}

void MyString::print()
{
    cout << pBuf << endl;
}

int MyString::getSize()
{
    return strlen(pBuf);
}

MyString MyString::operator+(MyString& s)
{
    char *temp = new char[getSize() + s.getSize() + 1];
    memset(temp, 0x00, getSize()+s.getSize()+1);
    memcpy(temp, pBuf, getSize());
    memcpy(&temp[getSize()], s.pBuf, s.getSize());
    MyString r(temp);
    delete [] temp;
    return r;
}
```

```
ostream& operator<<(ostream &output, const MyArray &a) {
    output << a.pBuf << endl;
    return output;
}

int main() {

    MyString s1("Hello ");
    MyString s2("World!");

    s1.print();
    s2.print();
    cout << "s1+s2=>" << (s1+s2) <<endl;

    return 0;
}
```

```
Hello
World!
Hello World!
```

연산자 중복과 프렌드 함수 예제

- 연산자 중복과 프렌드 함수를 이용한 MyArray 만들기

```
int main()
{
    MyArray a1(10);

    a1[0] = 1;
    a1[1] = 2;
    a1[2] = 3;
    a1[3] = 4;
    cout << a1 ;

    return 0;
}
```

1 2 3 4 0 0 0 0 0 0

연산자 중복과 프렌드 함수 예제 (계속)

```
#include <iostream>
#include <assert.h>
using namespace std;

// 항상된배열을나타낸다.
class MyArray {
    friend ostream& operator<<(ostream &, const MyArray &);    //
출력연산자<<
private:
    int *data;          // 배열의데이터
    int size;           // 배열의크기

public:
    MyArray(int size = 10);    // 디폴트생성자
    ~MyArray();               // 소멸자

    int getSize() const;      // 배열의크기를반환
    MyArray& operator=(const MyArray &a);    // = 연산자중복정의
    int& operator[](int i);    // [] 연산자중복: 설정자
};
```

```
MyArray::MyArray(int s) {
    size = (s > 0 ? s : 10);    // 디폴트크기를10으로한다.
    data = new int[size];    // 동적메모리할당

    for (int i = 0; i < size; i++)
        data[i] = 0;    // 요소들의초기화
}

MyArray::~MyArray() {
    delete [] data;    // 동적메모리반납
    data = NULL;
}

MyArray& MyArray::operator=(const MyArray& a) {
    if (&a != this) {    // 자기자신인자를체크
        delete [] data;    // 동적메모리반납
        size = a.size;    // 새로운크기를설정
        data = new int[size];    // 새로운동적메모리할당

        for (int i = 0; i < size; i++)
            data[i] = a.data[i];    // 데이터복사
    }
    return *this;    // a = b = c와같은경우를대비
}
```

```
int MyArray::getSize() const
{
    return size;
}

int& MyArray::operator[](int index) {
    assert(0 <= index && index < size);    // 인데스가범위에있지않으면종지
    return data[index];
}

// 프렌드함수정의
ostream& operator<<(ostream &output, const MyArray &a) {
    int i;
    for (i = 0; i < a.size; i++) {
        output << a.data[i] << ' ';
    }
    output << endl;
    return output;    // cout << a1 << a2 << a3와같은경우대비
}
```

```
int main()
{
    MyArray a1(10);

    a1[0] = 1;
    a1[1] = 2;
    a1[2] = 3;
    a1[3] = 4;
    cout << a1 ;

    return 0;
}
```

1 2 3 4 0 0 0 0 0 0